

# Scheduling

## 1. Gestión de datos y visualización:

- **struct Process:** Es la unidad básica de información. Almacena tanto los datos de entrada (burst, arrival) como los calculados (waiting, turnaround).
- **reset\_gantt():** Limpia el arreglo global gantt\_log llenándolo con -1. Esto es vital para que una simulación no "ensucie" a la siguiente.
- **draw\_gantt\_chart():** Traduce el arreglo gantt\_log a una representación visual en la consola.
- **get\_timestamp():** Utiliza las librerías de tiempo de C para generar una cadena de texto con la hora exacta del sistema, dándole realismo a los logs de "proceso iniciado/completado".

## 2. Algoritmos

### FIFO (First-In, First-Out):

**void simulate\_fifo(Process p[], int n)**

- **Lógica:** Primero ordena el arreglo por arrival\_time usando un ordenamiento de burbuja.
- **Ejecución:** Recorre los procesos en orden secuencial. Si la CPU está libre pero el proceso no ha llegado, adelanta el current\_time hasta el momento de su llegada. Ejecuta el proceso completo sin interrupciones.

### Round Robin (RR):

**void simulate\_rr(Process p[], int n, int quantum)**

- **Lógica:** Utiliza un enfoque cíclico. Mientras haya procesos sin terminar (completed < n), recorre el arreglo.
- **Bloque de Control:** Si un proceso ha llegado y le queda tiempo, se le asigna la CPU por un máximo de tiempo igual al quantum.
- **Actualización:** Al terminar su turno, se resta el tiempo ejecutado de su remaining\_time. Si llega a 0, se marca como completado.

### **SJF (Shortest Job First):**

**void simulate\_sjf(Process p[], int n)**

- **Lógica:** En cada unidad de tiempo (o al terminar un proceso), busca entre los que ya llegaron ( $\text{arrival\_time} \leq \text{current\_time}$ ) cuál tiene la ráfaga de tiempo más corta ( $\text{min\_burst}$ ).
- **Decisión:** Una vez seleccionado el más corto, se ejecuta hasta el final antes de volver a buscar.

### **SRTF (Shortest Remaining Time First):**

**void simulate\_srtf(Process p[], int n)**

- **Lógica:** En lugar de ejecutar el proceso completo, evalúa **segundo a segundo** qué proceso tiene el menor tiempo restante.
- **Expulsión:** Si mientras se ejecuta el proceso A, llega un proceso B con un tiempo de ráfaga menor al que le queda a A, la CPU cambia automáticamente a B.

### **3. Función Principal (main)**

**Inicialización:** Configura una semilla aleatoria y define cuántos procesos se crearán (entre 5 y 15).

**Generación:** Llena los datos iniciales de los procesos con valores aleatorios.

### **Ciclo de Simulación:**

- Crea una copia del dataset original usando memcpy. Esto es crucial porque los algoritmos modifican los valores de  $\text{remaining\_time}$  y  $\text{waiting\_time}$ .
- Llama a un algoritmo, muestra sus resultados, y reinicia la copia para el siguiente algoritmo.

### **Fórmulas Matemáticas**

Dentro de las funciones  $\text{print\_stats}$ , el código calcula automáticamente:

- **Tiempo de Retorno (Turnaround Time):**

$$TAT = \text{TIEMPO DE FINALIZACION} - \text{TIEMPO DE LLEGADA}$$

- **Tiempo de Espera (Waiting Time):**

$$WT = TAT - \text{TIEMPO DE RAFAGA}$$

